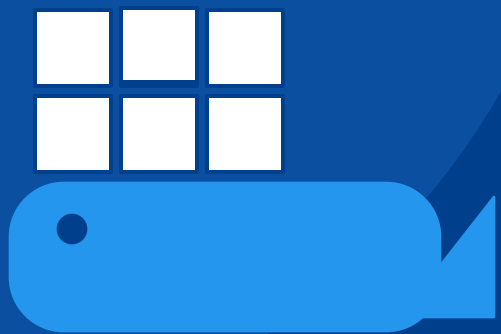




DOCKER

From Zero to Production

คู่มือเรียนรู้ Docker ตั้งแต่พื้นฐานจนใช้งานจริงสำหรับทีม IT & Developer

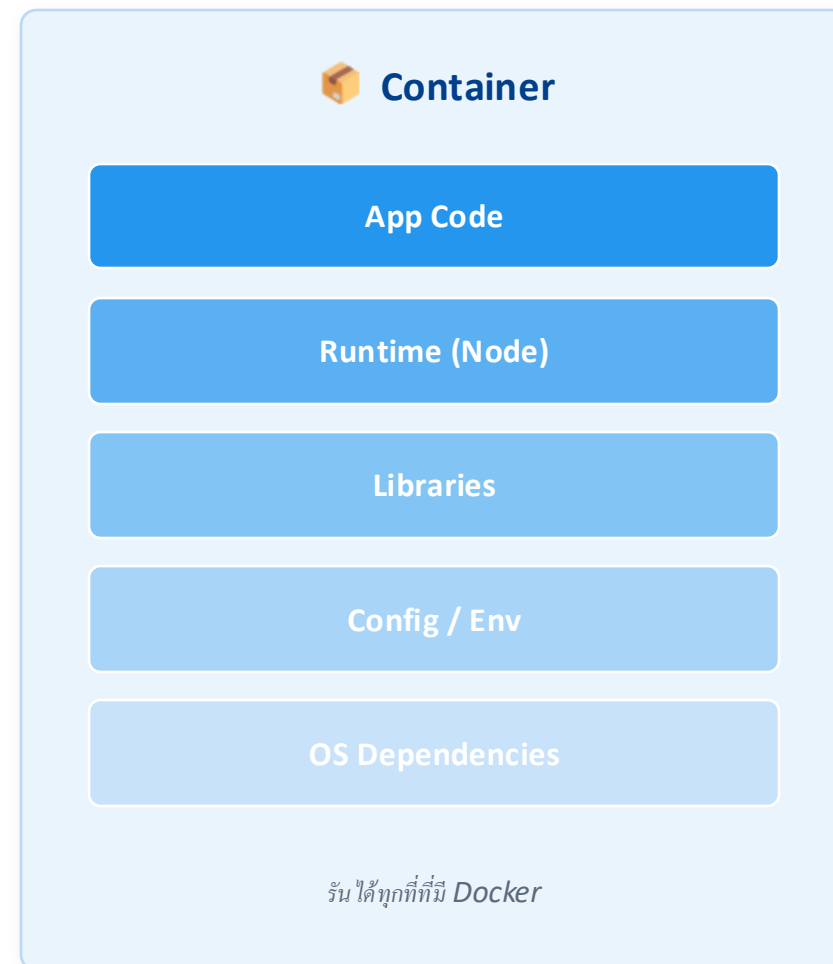
IT / Developer Training Series · 45 Slides · Beginner → Advanced

Docker คืออะไร?

Docker = เครื่องมือสำหรับ "แพ็ค" application พร้อมทุกสิ่งที่ต้องใช้ (code, library, dependency, config) ลงในกล่องเดี่ยวที่เรียกว่า **Container**

เปรียบเทียบง่ายๆ

- ส่งของทางเรือ → ใช้ "ตู้คอนเทนเนอร์" มาตรฐาน ขนของอะไรก็ได้
- Docker → ใช้ "กล่องสำเร็จรูป" รัน app อะไรก็ได้บนเครื่องไหนก็ได้
- เปิดกล่องแล้วใช้งานได้เลย ไม่ต้องตั้งค่าใหม่ทุกครั้ง
- กล่องเดียวกันรันเหมือนกัน บน laptop / server / cloud



ทำไมต้องใช้ Docker?

❌ ปัญหาแบบเดิม

"It works on my machine!"

- ติดตั้ง dependency ยาก, version เพี้ยน
- Setup server ใหม่ใช้เวลาเป็นชั่วโมง
- Dev / Staging / Prod ไม่เหมือนกัน

✅ ทางออกของ Docker

"ทำงานเหมือนกันทุกเครื่อง"

- Build ครั้งเดียว รัน ได้ทุกที่
- Setup ใหม่ภายในไม่กี่วินาที
- สิ่งแวดล้อมเหมือนกันตั้งแต่ Dev → Prod

5 ข้อดีหลัก



Portability

รันได้ทุกที่



Consistency

สภาพแวดล้อมเหมือนกัน



Fast Deploy

Deploy เร็วในไม่กี่วินาที



Isolation

แยกแต่ละ app ไม่ชนกัน

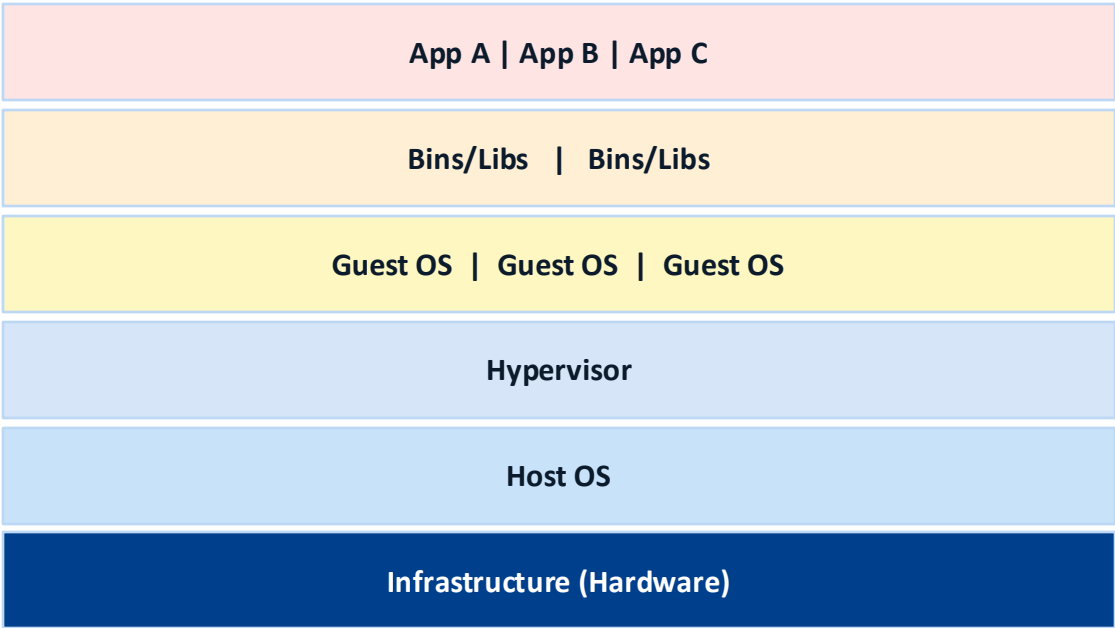


Efficient

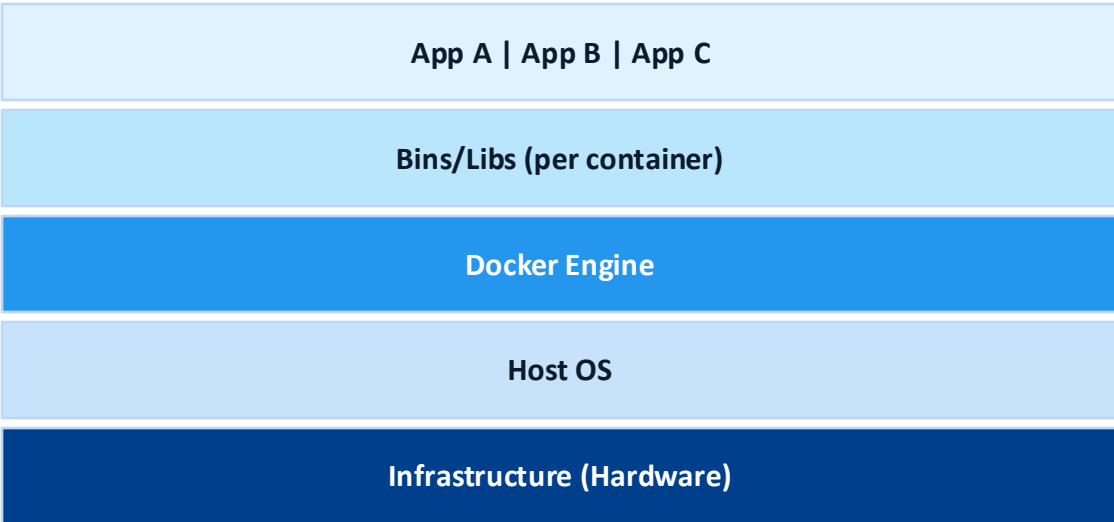
ใช้ resource น้อยกว่า VM

Docker vs Virtual Machine

Virtual Machine

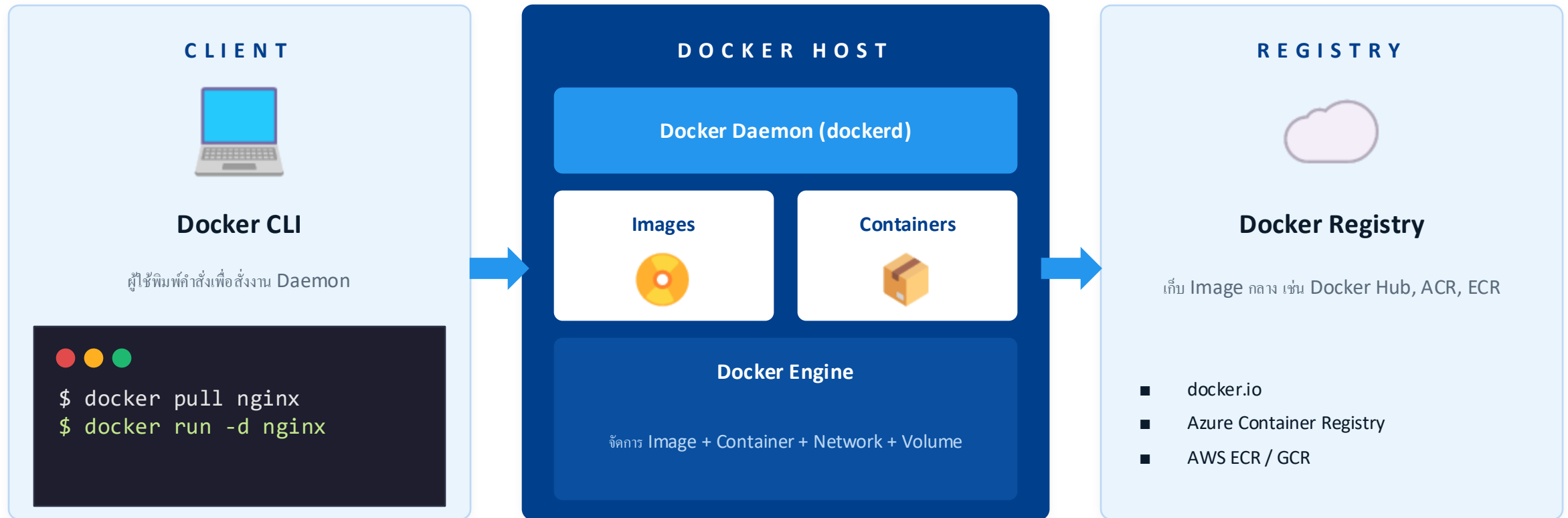


Docker Container



เปรียบเทียบ	Virtual Machine	Docker
ขนาด	GB (มี Guest OS)	MB (แชร์ Kernel)
เวลา Boot	นาที	วินาที
Isolation	เต็มรูปแบบ (แยก OS)	Process-level (เบากว่า)
Resource	ใช้มาก	ใช้น้อย

Docker Architecture



CLI ส่งคำสั่ง → Daemon ทำงาน → ดึง/ส่ง image จาก Registry

คำศัพท์สำคัญที่ต้องรู้จัก



Image

แบบพิมพ์ของ container — read-only template



Container

instance ที่รันอยู่จริงจาก Image



Dockerfile

ไฟล์สูตรสำหรับ build Image



Volume

ที่เก็บข้อมูลถาวร — ไม่หายเมื่อ container ถูกลบ



Network

ตัวเชื่อม container ให้คุยกันได้



Registry

คลังกลางสำหรับเก็บ/แชร์ Image



Compose

เครื่องมือรันหลาย container พร้อมกัน

Docker Hub คืออะไร?



Public Registry กลางของ Docker — มี image พร้อมใช้นับล้านตัว

- hub.docker.com — ฟรี search + pull
- Official Images: nginx, node, mysql, redis, postgres
- Verified Publisher: image จากบริษัทที่เชื่อถือได้
- Private Repository: เก็บ image ของทีมแบบ private

วิธี pull image

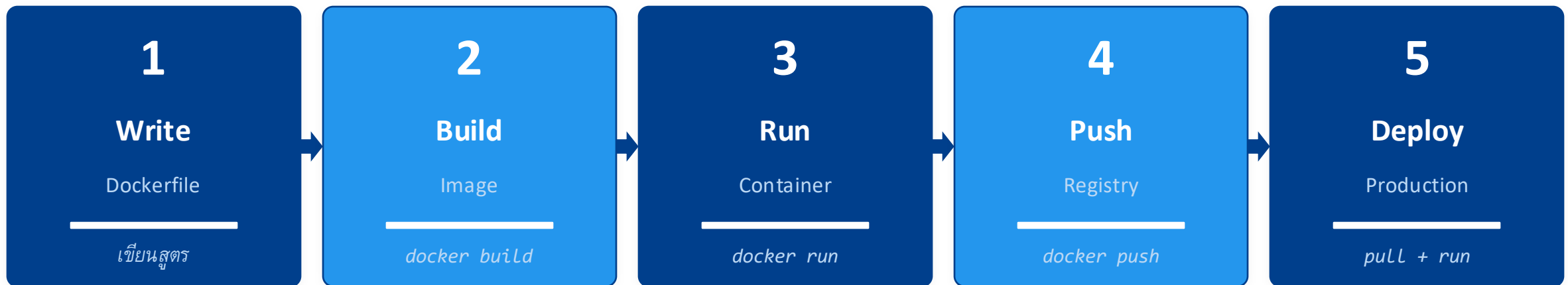
```
● ● ●  
# ดึง image จาก Docker Hub  
$ docker pull nginx:latest  
$ docker pull node:20-alpine  
  
# ค้นหา image  
$ docker search redis
```

Tag = เวอร์ชัน เช่น nginx:1.25, node:20-alpine



ระวัง! Image ที่ไม่ใช่ Official อาจมี vulnerability — ควรใช้ Official หรือ scan ก่อนใช้เสมอ

Docker Workflow Overview



Cycle เดิม ทำซ้ำได้ทุกครั้งที่เรา release version ใหม่



```
$ docker build -t myapp:1.0 . → $ docker run -p 80:3000 myapp:1.0 → $ docker push myapp:1.0
```

System Requirements

OS	เวอร์ชันที่รองรับ	RAM	CPU	หมายเหตุ
 Windows 10 / 11	Pro / Ent / Edu (64-bit)	ขั้นต่ำ 4 GB	x64 / ARM64	ต้องเปิด WSL2 + Virtualization
 macOS	11 (Big Sur) ขึ้นไป	ขั้นต่ำ 4 GB	Intel / Apple Silicon	ใช้ Docker Desktop
 Linux Ubuntu	20.04 / 22.04 / 24.04	ขั้นต่ำ 2 GB	x64 / ARM	ติดตั้งผ่าน apt
 Linux RHEL/CentOS	RHEL 8+, Rocky 9+	ขั้นต่ำ 2 GB	x64	ใช้ dnf / yum

ที่แนะนำสำหรับใช้งานจริง



RAM 8 GB+

สำหรับรัน DB + app พร้อมกัน



CPU 4 cores+

Multi-stage build เร็วขึ้น



Disk 20 GB+

Image / Volume กินที่



Internet

ใช้ pull image จาก Registry

ติดตั้ง Docker Desktop บน Windows

1

Download

ดาวน์โหลดจาก docker.com/products/docker-desktop

2

Install

ดับเบิลคลิก installer → คลิก "Use WSL2"

3

WSL2

เปิด PowerShell (Admin): `wsl --install`

4

Restart

Restart เครื่องหลังติดตั้งเสร็จ

5

Sign In

เปิด Docker Desktop → login (optional)

6

Test

ลองรัน `docker --version` ใน Terminal

 **Screenshot Placeholder***Docker Desktop Dashboard*

Docker Desktop

✓ Engine running

คำสั่งเปิด WSL2



```
# PowerShell (Run as Administrator)
PS> wsl --install
PS> wsl --set-default-version 2
PS> docker --version
```

ติดตั้ง Docker บน Linux (Ubuntu)

ติดตั้งทีละขั้น



```
# 1) อัปเดต package list
$ sudo apt-get update

# 2) ติดตั้ง dependencies และ Docker Engine
$ sudo apt-get install -y ca-certificates curl gnupg
$ curl -fsSL https://get.docker.com | sudo sh

# 3) เพิ่ม user เข้า docker group (ไม่ต้องใช้ sudo ทุกครั้ง)
$ sudo usermod -aG docker $USER
$ newgrp docker

# 4) เริ่มและเปิดอัตโนมัติเมื่อ boot
$ sudo systemctl enable --now docker
```



ตรวจสอบ: `docker run hello-world`



ถ้า newgrp ไม่ work — logout/login แล้วลองใหม่

ตรวจสอบการติดตั้ง



```
$ docker --version  
Docker version 25.0.3, build 4debf41
```

ดูเวอร์ชันของ *Docker Engine* ที่ลงไว้



```
$ docker info  
Containers: 0   Images: 0   ...
```

ดูข้อมูลโดยรวมของ *Daemon* (containers, images, storage)

Run hello-world ทดสอบครั้งแรก



```
$ docker run hello-world  
Unable to find image 'hello-world:latest' locally  
latest: Pulling from library/hello-world  
Status: Downloaded newer image for hello-world:latest
```

Hello from Docker!

This message shows that your installation appears to be working correctly.

คำสั่งพื้นฐานที่ต้องรู้

Command	ใช้ทำอะไร	ตัวอย่าง
docker pull	ดึง image จาก registry	docker pull nginx
docker run	สร้างและรัน container ใหม่	docker run -d nginx
docker ps	ดู container ที่กำลังรัน (-a = ทั้งหมด)	docker ps -a
docker stop	หยุด container ที่กำลังรัน	docker stop web
docker start	เริ่ม container ที่หยุดอยู่	docker start web
docker rm	ลบ container	docker rm web
docker images	ดูรายการ image บนเครื่อง	docker images
docker rmi	ลบ image	docker rmi nginx
docker logs	ดู log ของ container	docker logs -f web
docker exec	รันคำสั่งใน container	docker exec -it web sh

💡 พิมพ์ `docker <command> --help` เพื่อดู *option* ทั้งหมด

รัน Container แรกของคุณ — Nginx Web Server

เป้าหมาย: รัน Nginx ฟัง port 80 บนเครื่องของคุณ



```
$ docker run -d -p 80:80 --name web nginx
```

-d

Detached mode — รันเบื้องหลัง (ไม่ block terminal)

-p 80:80

Map port host:container — เปิด port 80 ของเครื่องไปที่ 80 ของ container

--name web

ตั้งชื่อ container ว่า "web" (แทน random name)

nginx

ชื่อ image (Docker Hub) — สิ่งอัตโนมัติถ้าไม่มี

ผลลัพธ์

- เปิด browser → <http://localhost> เห็นหน้า Welcome to nginx!
- `docker ps` → เห็น container ชื่อ web รันอยู่

การจัดการ Container

**start**`docker start web`

เริ่ม container ที่หยุดอยู่

**restart**`docker restart web`

หยุดแล้วเริ่มใหม่

**logs**`docker logs -f web`

ดู log แบบ realtime

**stop**`docker stop web`

หยุดอย่างปลอดภัย (SIGTERM)

**rm**`docker rm web`

ลบ container (ต้องหยุดก่อน)

**exec**`docker exec -it web sh`

เข้า shell ใน container

**Tip**

`docker exec -it <name> bash` ใช้กับ image ที่มี bash, sh ใช้กับ alpine; -it = interactive + tty

การจัดการ Image

pull	ดึง image จาก registry	<code>docker pull node:20-alpine</code>
build	สร้าง image จาก Dockerfile	<code>docker build -t myapp:1.0 .</code>
tag	ตั้ง tag ให้ image	<code>docker tag myapp:1.0 user/myapp:1.0</code>
push	ส่งขึ้น registry	<code>docker push user/myapp:1.0</code>
images	ดูรายการ image + ขนาด	<code>docker images</code>
rmi	ลบ image	<code>docker rmi myapp:1.0</code>

ตัวอย่าง **output** ของ **docker images**

				
REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
myapp	1.0	a1b2c3d4e5f6	2 hours ago	142MB

เขียน Dockerfile แรก



```
# Dockerfile (Node.js app)
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./
RUN npm ci --omit=dev

COPY . .

EXPOSE 3000

CMD ["node", "server.js"]
```

FROM

เลือก base image (มี Node + Alpine Linux)

WORKDIR

ตั้ง directory ที่ใช้ทำงานใน container

COPY (1)

คัดลอก package.json ก่อน — ใช้ cache layer

RUN

ติดตั้ง dependency ตอน build

COPY (2)

คัดลอก source code ที่เหลือ

EXPOSE

ประกาศ port (เอกสาร — ไม่เปิดจริง)

CMD

คำสั่งเริ่มต้นเมื่อ container ถูกรัน



```
$ docker build -t myapp:1.0 . && $ docker run -d -p 3000:3000 myapp:1.0
```

Dockerfile Best Practices



Multi-stage Build

แยก build/runtime — image เล็กลงมาก



.dockerignore

กัน node_modules, .git, .env เข้าไปใน image



ไม่ run as root

ใช้ USER node — ลด attack surface



ลด Layer

รวม RUN หลาย step ให้เป็น chain ด้วย &&



Alpine Base

base image เล็ก (~5MB) เร็วและปลอดภัย



Pin Version

ใช้ node:20.10-alpine ไม่ใช่ :latest

⚠ อย่าใส่ secret ลงใน image — ใช้ environment variable หรือ secret manager แทน

Build Image จาก Node.js + Express App

Dockerfile

```
FROM node:20-alpine
WORKDIR /app

COPY package*.json ./
RUN npm ci --omit=dev

COPY . .

USER node
EXPOSE 3000

CMD ["node", "index.js"]
```

ขั้นตอนการ Build & Run

```
# Build image
$ docker build -t express-app:1.0 .

# Run container
$ docker run -d -p 3000:3000 \
  --name api express-app:1.0
```



ทดสอบ

- docker logs api → server listening on 3000
- curl http://localhost:3000 → ได้ JSON response
- docker images → เห็น express-app:1.0

Docker Volumes — เก็บข้อมูลถาวร



Named Volume

Docker จัดการให้ — recommended

```
-v myvol:/data
```

ใช้เมื่อไหร่

Database, Production data



Bind Mount

ผูกกับ folder จริงบน host

```
-v ./code:/app
```

ใช้เมื่อไหร่

Development (live reload)



tmpfs

อยู่ใน RAM อย่างเดียว ไม่ถูกบันทึก

```
--tmpfs /tmp
```

ใช้เมื่อไหร่

Cache, secrets ชั่วคราว

Container $\rightleftarrows$ Volume Mount Point $\rightleftarrows$ Host File System / Memory

Docker Network

bridge

Default — แต่ละ container แยกอยู่ใน virtual network

host

ใช้ network ของ host โดยตรง (ไม่มี isolation)

none

ไม่มี network เลย — สำหรับ container ที่ไม่ต้องการเชื่อมต่อ

custom

สร้างเอง — recommended สำหรับ Compose, แยก app



```
# ใช้ custom network
$ docker run -d --network app-net \
  --name app myapp:1.0
```

Custom Bridge Network: app-net

**app****db****cache**

เชื่อมกันด้วยชื่อ container เป็น hostname



```
$ docker network create app-net
```

Environment Variables & Secrets

3 วิธีส่ง environment variable

1) ตัวเดียว — -e

```
$ docker run \
  -e DB_HOST=db \
  -e PORT=3000 \
  myapp
```

2) จากไฟล์ — --env-file

```
# .env
DB_HOST=db
PORT=3000
$ docker run --env-file .env myapp
```

3) Docker Secret (Swarm)

```
$ echo "pass" | \
  docker secret create db_pwd -
# read at /run/secrets/db_pwd
```

⚠️ ข้อควรระวังเรื่อง Secrets

- ห้าม! commit ไฟล์ .env ขึ้น Git — ใส่ใน .gitignore เสมอ
- ห้าม! ใส่ secret ใน Dockerfile (FROM/RUN/ENV) — ติดอยู่ใน image
- ใช้ Secret Manager ใน Production: Azure Key Vault, AWS Secrets Manager, Vault
- ตรวจสอบ image leak ด้วย docker history + scan tool ก่อน push

Docker Compose คืออะไร?

เครื่องมือสำหรับ "นิยาม + รันหลาย container" ด้วยไฟล์ YAML ไฟล์เดียว

✗ Without Compose

```
$ docker network create app-net
$ docker volume create db-data
$ docker run -d --name db \
  --network app-net \
  -v db-data:/var/lib/mysql ...
$ docker run -d --name app \
  --network app-net \
  -e DB_HOST=db ...
# ยาวมาก ลืมง่าย ทำซ้ำได้ยาก
```

✓ With Compose

```
# docker-compose.yml
services:
  app:
    build: .
  db:
    image: mysql:8

$ docker compose up -d
# 1 คำสั่ง – รันทุก container พร้อมใช้!
```

ใช้เมื่อ: **app** มีหลายส่วน (*frontend + backend + DB + cache*)

โครงสร้าง docker-compose.yml

```
version: "3.9"

services:
  web:
    build: .
    ports: ["3000:3000"]
    environment:
      DB_HOST: db
    depends_on: [db]
    networks: [app-net]
  db:
    image: mysql:8.0
    volumes: [db-data:/var/lib/mysql]

volumes: { db-data: {} }
networks: { app-net: {} }
```

version

เวอร์ชันของ Compose schema (3.9 ใหม่สุด)

services

รายการ container ที่จะรัน

build

สร้าง image จาก Dockerfile

image

ใช้ image สำเร็จรูป

ports

map host:container

environment

ENV variables

volumes

named volume / bind mount

networks

เลือก network

depends_on

ลำดับการ start

Compose สำหรับ Node.js + SQL Server

```
version: "3.9"
services:
  api:
    build: ./api
    ports: ["3000:3000"]
    environment:
      DB_HOST: db
      DB_USER: sa
      DB_PASS_FILE: /run/secrets/db_pwd
    depends_on: [db]
  db:
    image: mcr.microsoft.com/mssql/server:2022-latest
    environment:
      ACCEPT_EULA: "Y"
      SA_PASSWORD: "YourStrong!Passw0rd"
    volumes: [mssql-data:/var/opt/mssql]
    ports: ["1433:1433"]
volumes:
  mssql-data:
```

api build จาก ./api Dockerfile

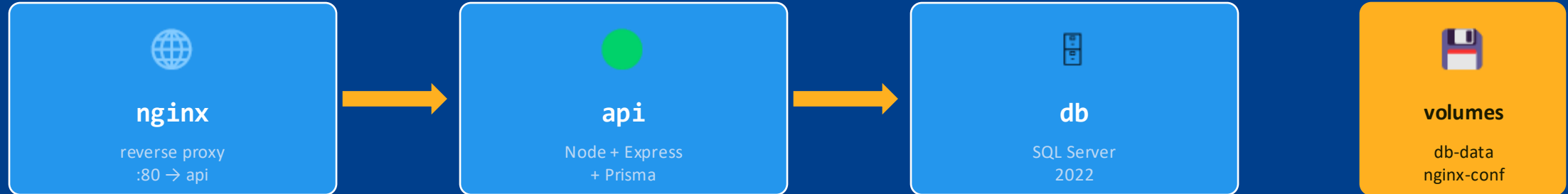
depends_on DB ขึ้นก่อน API

mssql-data volume เก็บ DB ถาวร

1433 port DB ของ SQL Server

```
$ docker compose up -d
$ docker compose logs -f api
$ docker compose down
```

Compose Full Stack: Nginx + Node + Prisma + SQL Server



```
services:
  nginx: { image: nginx:1.25-alpine, ports: ["80:80"], depends_on: [api],
    volumes: [./nginx.conf:/etc/nginx/nginx.conf:ro] }
  api: { build: ./api, environment: { DATABASE_URL: "sqlserver://db:1433;..." },
    depends_on: { db: { condition: service_healthy } } }
  db: { image: mcr.microsoft.com/mssql/server:2022-latest,
    environment: { ACCEPT_EULA: "Y", SA_PASSWORD: "..." },
    volumes: [db-data:/var/opt/mssql],
    healthcheck: { test: ["CMD", "/opt/mssql-tools/bin/sqlcmd", "..."] } }
volumes: { db-data: {} }
```

คำสั่ง Docker Compose ที่ใช้บ่อย



`docker compose up -d`

สร้าง + รัน services เบื้องหลัง



`docker compose logs -f`

ดู log แบบ realtime ทุก service



`docker compose exec api sh`

เข้า shell ใน service api



`docker compose ps`

ดูสถานะของแต่ละ service



`docker compose down`

หยุดและลบ container/network ทั้งหมด



`docker compose build`

build image ใหม่ตาม Dockerfile



`docker compose restart api`

restart service เดี่ยว



`docker compose config`

ตรวจ syntax ของ compose file

Compose Profiles & Override

Override File

Compose โหลด docker-compose.yml + docker-compose.override.yml อัปเดตอัตโนมัติ

```
# docker-compose.override.yml (dev)
services:
  api:
    volumes: [./api:/app]
    environment:
      NODE_ENV: development
    command: npm run dev
```

Profiles

รับบาง service เฉพาะตอนต้องการ เช่น tools, monitoring

```
services:
  pgadmin:
    image: dpage/pgadmin4
    profiles: [tools]

$ docker compose --profile tools up
```

✓ แนวทางใช้งานจริง

- docker-compose.yml = config กลาง (Production-like)
- docker-compose.override.yml = dev เฉพาะเครื่อง (live reload)
- docker-compose.prod.yml = ใช้กับ -f flag เมื่อขึ้น Production

Health Check & Restart Policy

healthcheck — ตรวจสอบว่า **container** พร้อมใช้งานจริง



```
services:
  api:
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3000/health"]
      interval: 30s   timeout: 5s   retries: 3   start_period: 20s
```

restart policy

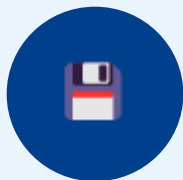
no	on-failure	always	unless-stopped
ไม่ restart (default)	restart เมื่อ exit code $\neq$ 0	restart เสมอ — แม้สั่ง stop	restart ยกเว้นถูกสั่ง stop
ใช้กับ: ใช้กับ <i>batch/cron</i>	ใช้กับ: <i>worker, retry job</i>	ใช้กับ: <i>service</i> สำคัญ	ใช้กับ: แนะนำสำหรับ <i>Production</i>

Compose Best Practices



Pin Image Version

ใช้ mysql:8.0.36 ไม่ใช่ :latest



Named Volumes

ไม่ใช่ bind mount ใน Production



แยก Network

frontend / backend / data tier แยก



ห้าม Hardcode Secret

ใช้ env_file + secret manager



ใช้ healthcheck

depends_on ด้วย service_healthy



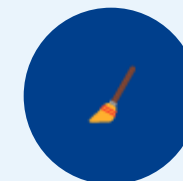
Build แยก stage

Multi-stage Dockerfile



กำหนด resources

limits cpu/memory ทุก service



Cleanup เป็นประจำ

docker system prune -a

Development vs Production

ด้าน	Development	Production
Volumes	bind mount: ./code:/app	named volume เท่านั้น
Restart	no / on-failure	unless-stopped / always
Build	build: . (build ทุกครั้ง)	image: registry/myapp:1.2.3
Logging	stdout / json-file	syslog / fluentd / centralized
Resources	ไม่จำกัด	limits CPU + memory เสมอ
Env Vars	.env file (local)	secret manager / vault
Hot Reload	✅ nodemon / watch	❌ build artifact ที่นิ่ง
NODE_ENV	development	production
Port Expose	เปิด DB ออก host	เปิดเฉพาะ proxy/web
Healthcheck	optional	บังคับ — ทุก service

Image Optimization สำหรับ Production

ก่อน (Single-stage)

1,024 MB

dev tools + source + node_modules + cache

หลัง (Multi-stage + Alpine)

102 MB

เฉพาะ runtime + production dependencies



```
# Stage 1: build
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . . && npm run build

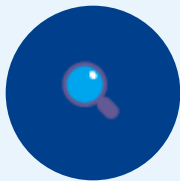
# Stage 2: runtime (เล็กกว่า 10x)
FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
USER node
CMD ["node", "dist/server.js"]
```

Security Best Practices



ไม่ run as root

USER node ใน Dockerfile



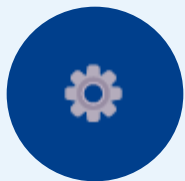
Scan vulnerabilities

docker scout cves / trivy image



Read-only FS

--read-only + tmpfs สำหรับ /tmp



Drop capabilities

--cap-drop=ALL --cap-add=NET_BIND



ไม่เก็บ secret ใน image

secret manager + secret mount



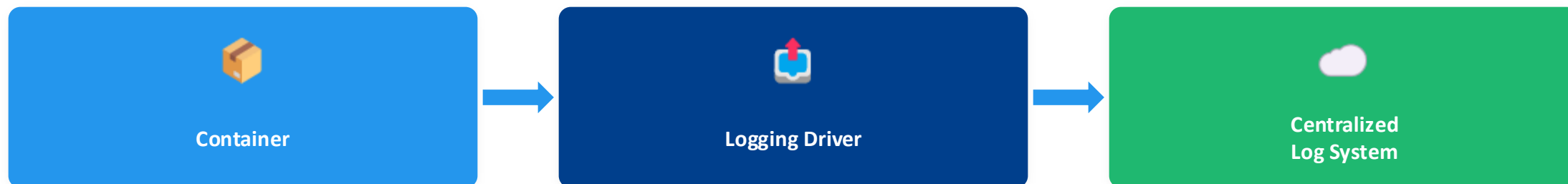
Pin base image

node:20.10.0-alpine ไม่ใช่ latest



```
# scan ก่อน push ทุกครั้ง
$ docker scout cves myapp:1.2.3
$ trivy image --severity HIGH,CRITICAL myapp:1.2.3
```

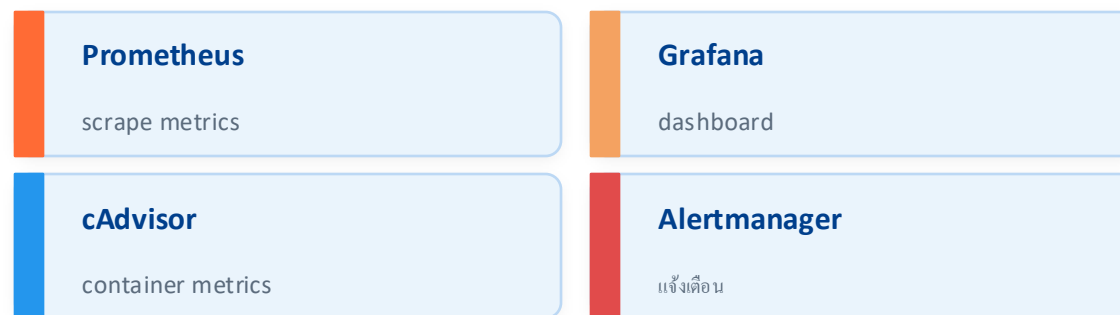
Logging & Monitoring



Logging Drivers ที่นิยม

- json-file (default) — เหมาะกับ dev
- syslog / journald — Linux native
- fluentd / gelf / awslogs — production
- splunk / loki — enterprise

Monitoring Stack



```
# กำหนด logging driver
$ docker run --log-driver=fluentd --log-opt fluentd-address=host:24224 myapp
```

Resource Limits

จำกัด CPU / Memory ป้องกัน container ตัวเดียวล้น host

1) Docker run

```
$ docker run -d \
  --cpus="1.5" \
  --memory="512m" \
  --memory-swap="1g" \
  myapp:1.0
```

2) Compose (deploy.resources)

```
services:
  api:
    deploy:
      resources:
        limits: { cpus: "1.5", memory: 512M }
        reservations: { cpus: "0.5", memory: 256M }
```

📌 แนวทางตั้งค่า

- เริ่มจาก measure จริงด้วย docker stats
- limits = ค่าสูงสุดที่ยอมให้ใช้ / reservations = ค่าที่จองไว้
- OOM Killer จะ kill process ถ้าใช้ memory เกิน limit
- DB ควรตั้ง memory limit สูง + reservation พอเหมาะ

Docker Registry ส่วนตัว



Docker Hub Private

ง่ายสุด, ราคาถูก

hub.docker.com — paid plan



Azure Container Registry

integration กับ Azure

myacr.azurecr.io



AWS ECR

integration กับ AWS / EKS

`aws ecr get-login-password`



Self-Hosted Registry

ฟรี ควบคุมเอง

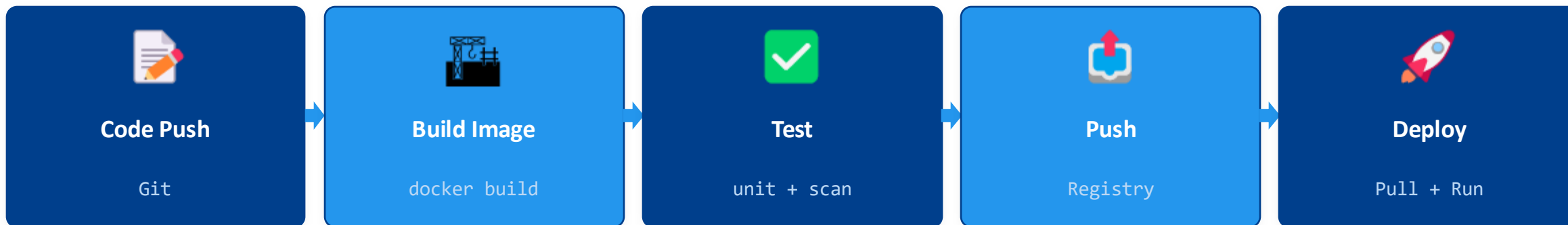
`docker run registry:2`



```
# 1) สร้าง self-hosted registry
$ docker run -d -p 5000:5000 --name registry registry:2

# 2) tag + push
$ docker tag myapp:1.0 myreg.local:5000/myapp:1.0
$ docker push myreg.local:5000/myapp:1.0
```

CI/CD Pipeline กับ Docker

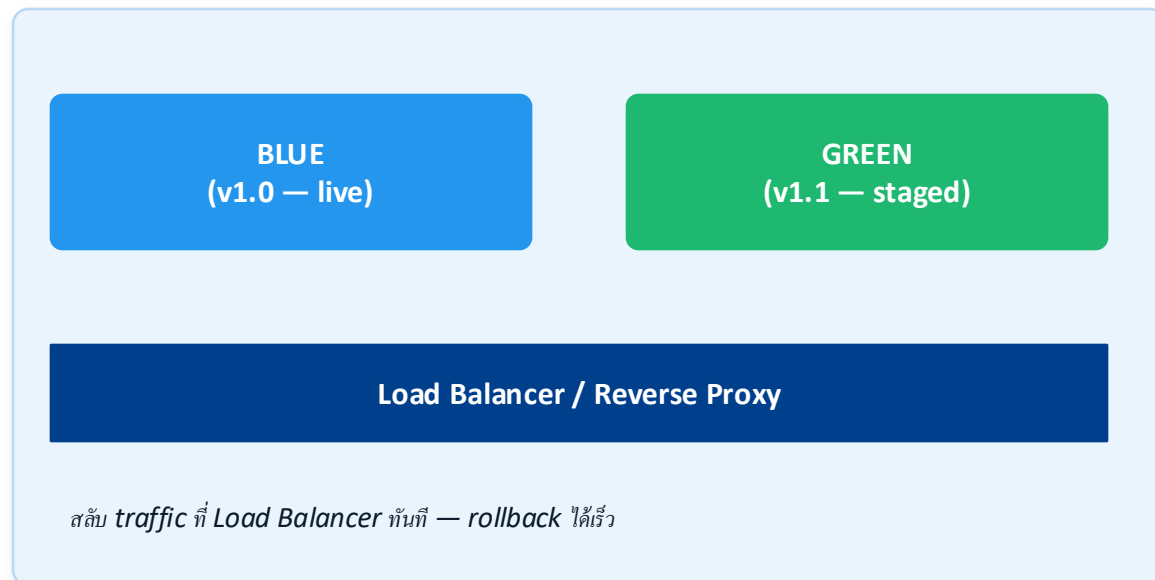


ตัวอย่าง **GitHub Actions**

```
# .github/workflows/docker.yml
name: Build & Push
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with: { username: ${secrets.DH_USER}}, password: ${secrets.DH_PWD} }
      - run: docker build -t user/myapp:${github.sha} .
      - run: docker push user/myapp:${github.sha}
```

Zero-Downtime Deployment

Blue-Green Deployment



Rolling Update



เลือกอย่างไร?

Blue-Green = rollback เร็วที่สุด แต่ใช้ infra 2 เท่า | Rolling = ประหยัด แต่ rollback ค่อยๆ — เลือกตาม SLA และงบ

Backup & Disaster Recovery



Volume Backup

tar archive volume → คัดลอกเก็บ



Database Dump

mysqldump / pg_dump → object storage



Image Snapshot

docker commit + push to registry



Off-site Storage

Azure Blob / S3 / Glacier

ตัวอย่าง backup volume



```
# backup
$ docker run --rm -v db-data:/data -v $(pwd):/backup alpine \
  tar czf /backup/db-data-$(date +%F).tar.gz -C /data .

# restore
$ docker run --rm -v db-data:/data -v $(pwd):/backup alpine \
  tar xzf /backup/db-data-2026-05-07.tar.gz -C /data
```

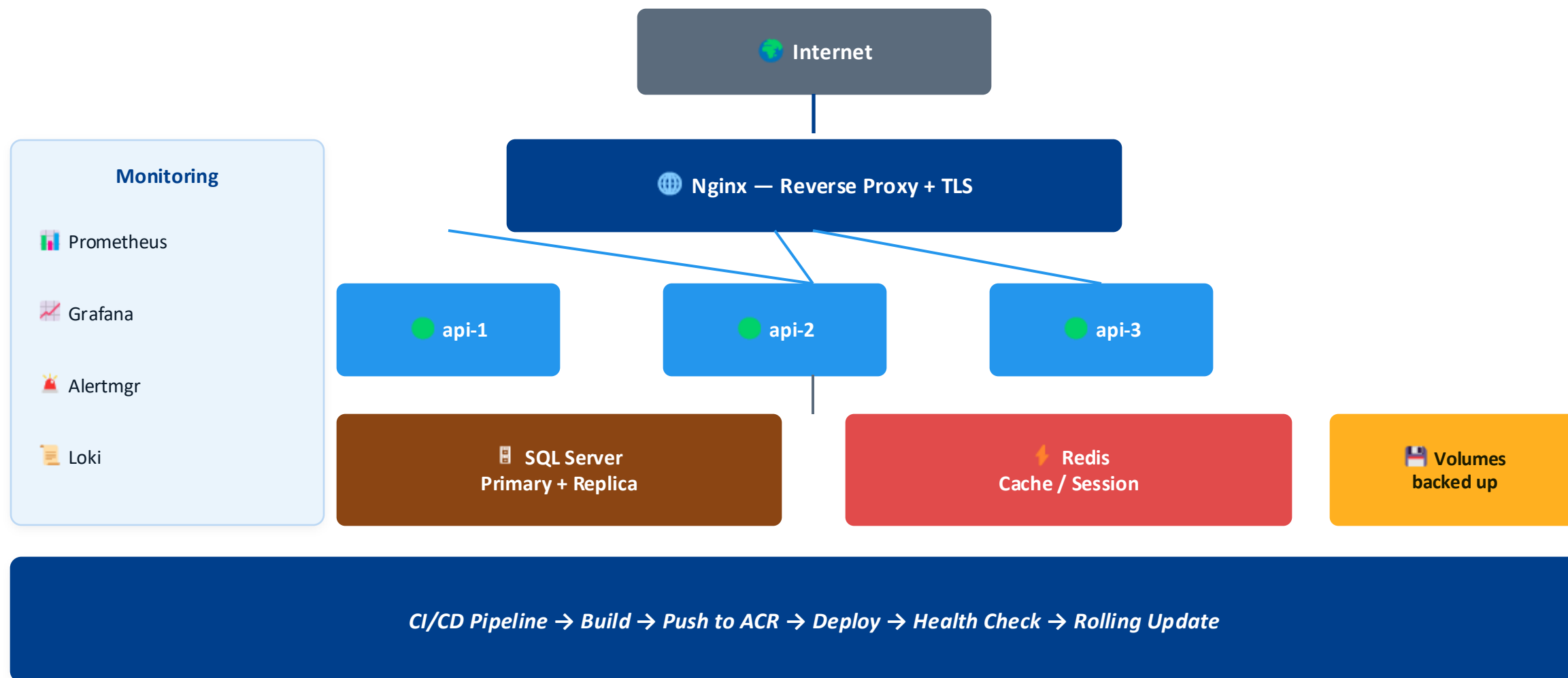
Docker Swarm vs Kubernetes (Overview)

ด้าน	Docker Swarm	Kubernetes (K8s)
Setup	ง่าย — built-in	ซับซ้อน — ต้อง config เยอะ
Learning Curve	ต่ำ	สูงชัน
Scaling	Manual / Auto basic	HPA + VPA + Cluster Autoscaler
Networking	เรียบง่าย	Service / Ingress / NetworkPolicy
Ecosystem	เล็ก	ใหญ่มาก (Helm, Operators, ฯลฯ)
Use case	ทีมเล็ก / โปรเจกต์ปานกลาง	Enterprise / scale ใหญ่

 เลือกอย่างไร?

- Swarm: ทีมเล็ก, ใช้ Docker อยู่แล้ว, ต้องการ orchestration ง่ายๆ
- K8s: ระบบขนาดใหญ่, multi-cloud, ต้องการ auto-scaling/observability ครบ
- ก่อนถึงทั้งสอง: Compose ก็เพียงพอสำหรับ small/medium production

Real-World Production Stack



Troubleshooting ปัญหาที่พบบ่อย

Container ไม่ start



`docker logs <c>` → ดู error
ตรวจ CMD / ENV / dependency

Volume Permission



Permission denied (rw)
`chown 1000:1000 /path` หรือ user: "1000"

Out of Disk Space



no space left on device
`docker system prune -a --volumes`

Port Conflict



Error: port already allocated
เปลี่ยน host port หรือ kill process เดิม

Image Pull Error



unauthorized / not found
`docker login` + ตรวจสอบ image:tag

Slow Build



Cache invalidate
เรียงลำดับ COPY ให้ leverage cache

Docker Command Cheat Sheet

IMAGE	CONTAINER	COMPOSE	SYSTEM
• <code>docker pull <image></code> • <code>docker build -t name:tag .</code> • <code>docker images</code> • <code>docker tag src dest</code> • <code>docker push <image></code> • <code>docker rmi <image></code>	• <code>docker run -d -p 80:80 </code> • <code>docker ps</code> / <code>docker ps -a</code> • <code>docker stop</code> / <code>start</code> / <code>restart</code> • <code>docker rm <name></code> • <code>docker logs -f <name></code> • <code>docker exec -it <name> sh</code>	• <code>docker compose up -d</code> • <code>docker compose down</code> • <code>docker compose logs -f</code> • <code>docker compose build</code> • <code>docker compose ps</code> • <code>docker compose exec <s> sh</code>	• <code>docker info</code> • <code>docker stats</code> • <code>docker system df</code> • <code>docker system prune -a</code> • <code>docker volume ls</code> / <code>prune</code> • <code>docker network ls</code>

Learning Path & Resources

Learning Path



Resources



Docker Official Docs

docs.docker.com



Docker Hub

hub.docker.com



Play with Docker

labs.play-with-docker.com



Docker Deep Dive

หนังสือโดย Nigel Poulton



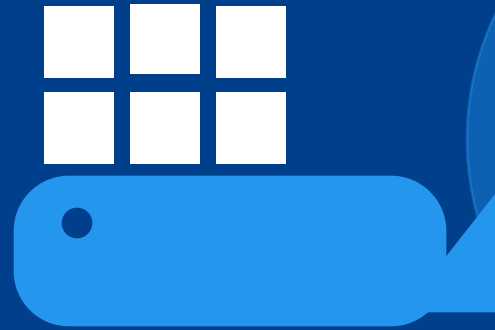
Docker Mastery

Bret Fisher (Udemy)



Awesome Docker

github.com/veggie-monk/awesome-docker



THANK YOU

Q & A

Happy Containerizing 🐳

✉️ it-training@company.com
💬 [#docker-help](#) • 🌐 [wiki/docker](#)